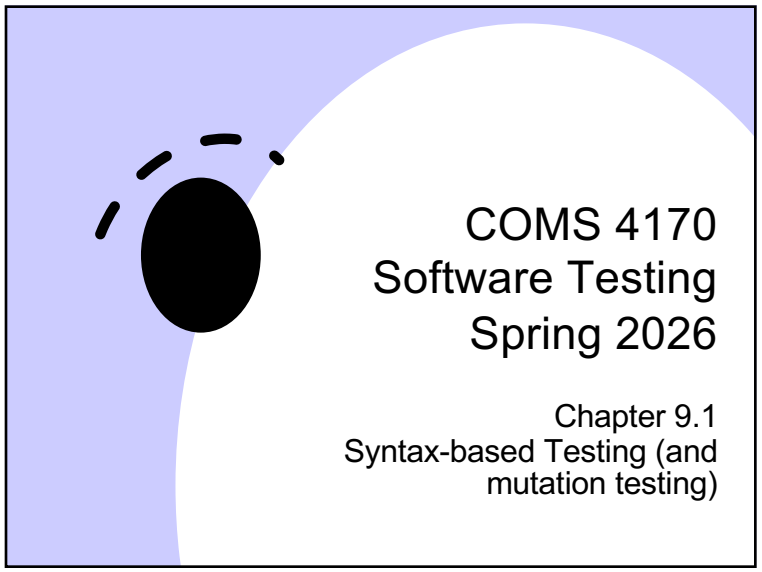
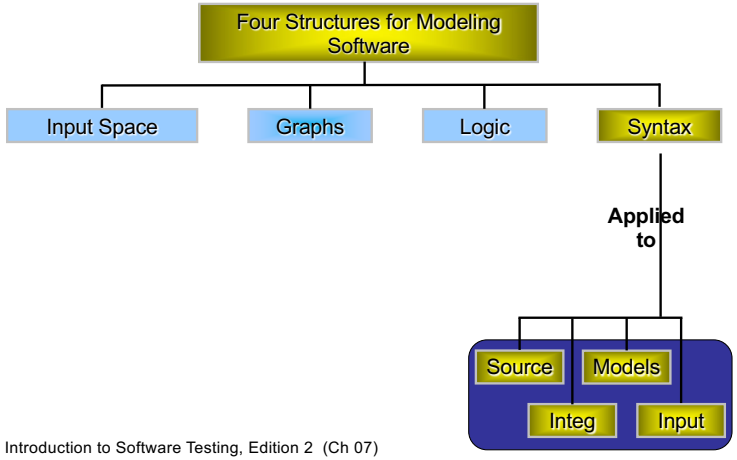


COMS 4170 Software Testing Spring 2026

Chapter 9.1
Syntax-based Testing (and mutation testing)

1

Ch. 9: Syntax Coverage



2

Using the Syntax to Generate Tests

- Lots of software artifacts follow strict syntax rules
- The syntax is often expressed as a grammar in a language such as BNF
- Syntactic descriptions can come from many sources
  - Programs
  - Integration elements
  - Design documents
  - Input descriptions
- Tests are created with two general goals
  - Cover the syntax in some way
  - Violate the syntax (invalid tests)

3

Grammar Coverage Criteria

- Software engineering makes practical use of automata theory in several ways
  - Programming languages defined in BNF
  - Program behavior described as finite state machines
  - Allowable inputs defined by grammars
- A simple regular expression: ***(G s n | B t n)****

  '*' is *closure* operator, zero or more occurrences

  '|' is *choice*, either one can be used
- Any sequence of "*G s n*" and "*B t n*"
- '*G*' and '*B*' could represent commands, methods, or events
- '*s*', '*t*', and '*n*' can represent arguments, parameters, or values
- '*s*', '*t*', and '*n*' could represent literals or a set of values

4

Test Cases from Grammar

- A string that satisfies the derivation rules is said to be "*in the grammar*"
- A test case is a sequence of strings that satisfy the regular expression
- Suppose 's', 't' and 'n' are numbers

```
G 26 08 01 90
B 22 06 27 94
G 22 11 21 94
B 13 01 09 03
```

Could be one test with four parts or four separate tests, etc.

5

BNF Grammars

Stream ::= action*

action ::= actG | actB

actG ::= "G" s n

actB ::= "B" t n

s ::= digit$^{1-3}$

t ::= digit$^{1-3}$

n ::= digit^{2} "." digit^{2} "." digit^{2}

digit ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Start symbol

Non-terminals

Production rule

Terminals

6

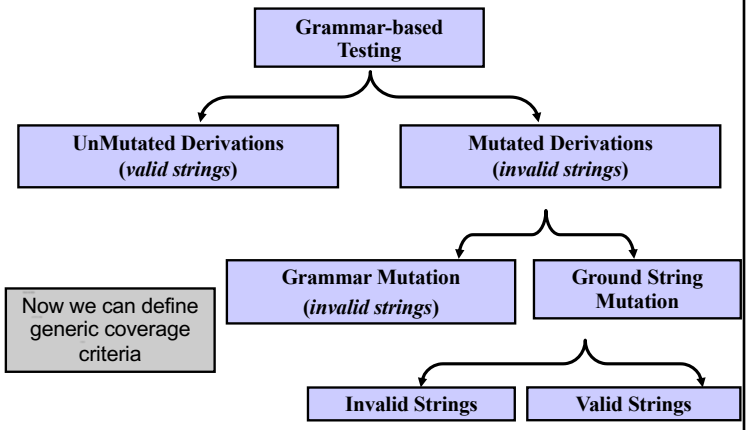
Using Grammars

Stream ::= action action *
::= actG action*
::= G s n action*
::= G digit$^{1-3}$ digit^{2} . digit^{2} . digit^{2} action*
::= G digitdigit digitdigit.digitdigit.digitdigit action*
::= G 25 08.01.90 action*
...

- Recognizer : Is a string (or test) in the grammar ?
  - This is called parsing
  - Tools exist to support parsing
  - Programs can use them for input validation
- Generator : Given a grammar, derive strings in the grammar

7

Mutation as Grammar-Based Testing



8

Grammar-based Coverage Criteria (9.1.1)

- The most common and straightforward criteria use every terminal and every production at least once

Terminal Symbol Coverage (TSC) : TR contains each terminal symbol *t* in the grammar *G*.

Production Coverage (PDC) : TR contains each production *p* in the grammar *G*.

9

Grammar-based Coverage Criteria

- PDC subsumes TSC
- Grammars and graphs are interchangeable
  - PDC (production coverage) is equivalent to EC (edge coverage), TSC (terminal symbol coverage) is equivalent to NC (node coverage)
- Other graph-based coverage criteria could be defined on grammar

10

Grammar-based Coverage Criteria

- A related criterion is the impractical one of deriving all possible strings

Derivation Coverage (DC) : TR contains every possible string that can be derived from the grammar *G*.

11

Grammar-based Coverage Criteria

- The number of TSC tests is bound by the number of terminal symbols
  - 13 in the stream grammar
- The number of PDC tests is bound by the number of productions
  - 18 in the stream grammar
- The number of DC tests depends on the details of the grammar
  - 2,000,000,000 in the stream grammar!
- All TSC, PDC and DC tests are in the grammar … how about tests that are NOT in the grammar ?

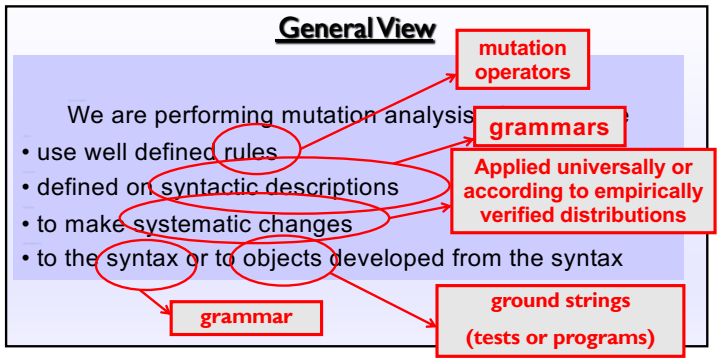
12

Mutation Testing (9.1.2)

- Grammars describe both valid and invalid strings
- Both types can be produced as mutants
- A mutant is a variation of a valid string
  - Mutants may be valid or invalid strings
- Mutation is based on "mutation operators" and "ground strings"

13

What is Mutation ?



14

Mutation Testing

- Ground string: A string in the grammar
  - The term "ground" is used as an analogy to algebraic ground terms
- Mutation Operator : A rule that specifies syntactic variations of strings generated from a grammar
- Mutant: The result of one application of a mutation operator
  - A mutant is a string either in the grammar or very close to being in the grammar

15

Mutants and Ground Strings

- The key to mutation testing is the design of the mutation operators
  - Well designed operators lead to powerful testing
- Sometimes mutant strings are based on ground strings
- Sometimes they are derived directly from the grammar
  - Ground strings are used for valid tests
  - Invalid tests do not need ground strings

Valid Mutants	
Ground Strings	Mutants
G 26 08.01.90	B 26 08.01.90
B 22 06.27.94	B 45 06.27.94

Invalid Mutants
7 26 08.01.90
B 22 06.27.1

16

```
Stream  ::= action*
action  ::= actG | actB
actG    ::= "G" s n
actB    ::= "B" t n
s       ::= digit[1-3]
t       ::= digit[1-3]
n       ::= digit[2] "." digit[2] "." digit[2]
digit   ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" |
            "7" | "8" | "9"
```

Valid Mutants

Ground Strings	Mutants
G 26 08.01.90	B 26 08.01.90
B 22 06.27.94	B 45 06.27.94

Invalid Mutants

7 26 08.01.90

B 22 06.27.1

17

Questions About Mutation

- Should more than one operator be applied at the same time?
- Should a mutated string contain more than one one mutated element?
  - Usually not – multiple mutations can interfere with each other
  - Experience with program-based mutation indicates not
  - Recent research is finding exceptions

18

Questions About Mutation

- Should every possible application of a mutation operator be considered?
  - Necessary with program-based mutation

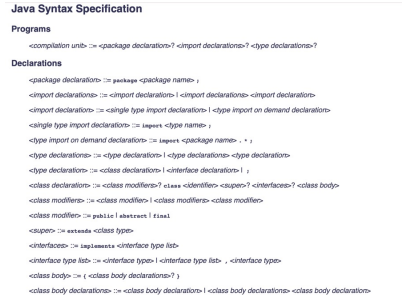
19

Mutation Testing Generality

- Mutation operators have been defined for many languages
  - Programming languages (*Fortran, Lisp, Ada, C, C++, Java, Python*)
  - Specification languages (*SMV, Z, Object-Z, algebraic specs*)
  - Modeling languages (*Statecharts, activity diagrams*)
  - Input grammars (*XML, SQL, HTML*)

20

Example BNF for Java



Java Syntax Specification

Programs

<compilation unit> ::= <package declaration>? <import declarations>? <type declarations>?

Declarations

<package declaration> ::= `package` <package name> ;

<import declarations> ::= <import declaration> | <import declarations> <import declaration>

<import declaration> ::= <single type import declaration> | <type import on demand declaration>

<single type import declaration> ::= `import` <type name> ;

<type import on demand declaration> ::= `import` <package name> . * ;

<type declarations> ::= <type declaration> | <type declarations> <type declaration>

<type declaration> ::= <class declaration> | <interface declaration> | ;

<class declaration> ::= <class modifiers>? `class` <identifier> <super>? <interfaces>? <class body>

<class modifiers> ::= <class modifier> | <class modifiers> <class modifier>

<class modifier> ::= `public` | `abstract` | `final`

<super> ::= `extends` <class type>

<interfaces> ::= `implements` <interface type list>

<interface type list> ::= <interface type> | <interface type list> , <interface type>

<class body> ::= { <class body declarations>? }

<class body declarations> ::= <class body declaration> | <class body declarations> <class body declaration>

Anders Møller
https://cs.au.dk/~amoeller/RegAut/JavaBNF.html

21

What is a Mutant?

Mutant

A small syntactic change to a programming artifact

(program, statechart, XML, SQL, specification, …)

Mutation operators are defined on the underlying grammar

(change an operator to another compatible operator, change an edge from one target state to another, …)

22

Killing Mutants

- When ground strings are mutated to create valid strings, the hope is to exhibit different behavior from the ground string
- This is normally used when the grammars are programming languages, the strings are programs, and the ground strings are pre-existing programs

23

Killing Mutants

- Killing Mutants: Given a mutant $m \in M$ for a derivation D and a test t, t is said to kill m if and only if the output of t on D is different from the output of t on m
- The derivation D may be represented by the list of productions or by the final string

24

Syntax-based Coverage Criteria

- Coverage is defined in terms of killing mutants

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m.

- Coverage in mutation equates to number of mutants killed
- The ratio of mutants killed is called the mutation score

Syntax-based Coverage Criteria

- When creating invalid strings, we just apply the operators
- This results in two simple criteria
- It makes sense to either use every operator once or

Mutation Operator Coverage (MOC) : For each mutation operator, TR contains exactly one requirement, to create a mutated string m that is derived using the mutation operator.

Mutation Production Coverage (MPC) : For each mutation operator, TR contains several requirements, to create one mutated string m that includes every production that can be mutated by that operator.

Example

Grammar

```
Stream  ::= action*
action  ::= actG | actB
actG    ::= "G" s n
actB    ::= "B" t n
s       ::= digit[1-3]
t       ::= digit[1-3]
n       ::= digit[2] "." digit[2] "." digit[2]
digit   ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Ground String

G 25 08.01.90

B 21 06.27.94

Mutation Operators

- *Exchange actG and actB*
- *Replace digits with all other digits*

Mutants using MOC

B 25 08.01.90

B 23 06.27.94

Mutants using MPC

B 25 08.01.90	*G 21 06.27.94*
G 15 08.01.90	*B 22 06.27.94*
G 35 08.01.90	*B 23 06.27.94*
G 45 08.01.90	*B 24 06.27.94*
…	…

Mutation Testing

- The number of test requirements for mutation depends on two things
  - The syntax of the artifact being mutated
  - The mutation operators
- Mutation testing is very difficult to apply by hand
- Mutation testing is very effective – considered the "gold standard" of testing
- Mutation testing is often used to evaluate other criteria

Program-based Mutation

- The original and most widely known application of mutation is to modify programs
- Operators modify a ground string (program under test) to create mutant programs
- Mutant programs must compile correctly (must be valid strings in the grammar)
- Once mutants are defined, tests must be found to cause mutants to fail
- This is called "killing mutants"

Categorizing Mutants

Testers can keep adding tests until all mutants have been killed

Dead mutant : A test case has killed it

Trivial mutant : Almost every test can kill it

Equivalent mutant : No test can kill it (same behavior as original)

Program Mutation Example

Original Method

```
int Min (int A, int B)
{
      int minVal;
      minVal = A;
      if (B < A)
      {
          minVal = B;
       }
       return (minVal);
} // end Min
```

6 mutants

Each represents a separate program

With Embedded Mutants

```
int Min (int A, int B)
{
      int minVal;
      minVal = A;
Δ 1   minVal = B;
      if (B < A)
Δ 2   if (B <= A)
Δ 3   if (B < minVal)
      {
          minVal = B;
Δ 4       Bomb ();
Δ 5       minVal = A;
Δ 6       minVal = failOnZero (B);
      }
      return (minVal);
} // end Min
```

Replace one variable with another

Replaces operator

Immediate runtime failure … if reached

Immediate runtime failure if B==0, else does nothing

Equivalent Mutation Example

- Mutant 3 in the Min() example is equivalent:

```
      minVal = A;
      if (B < A)
Δ 3   if (B < minVal)
```

- The mutant can only be killed if (B < A) != (B < minVal)
- However, the previous statement was "minVal = A"
  - Substituting, we get: "(B < A) != (B < A)"
  - This is a logical contradiction !
- Thus no input can kill this mutant

Mutation and RIPR

- The RIPR model :
  - *Reachability*: The test causes the ***mutated*** statement to be reached
  - *Infection*: The test causes the ***mutated*** statement to result in an incorrect state
  - *Propagation*: The incorrect state propagates to incorrect output
  - *Revealability*: The tester must observe part of the incorrect output
- The RIPR model leads to two variants of mutation coverage …
  1. Strong mutation: propagation is required
  2. Weak mutation: only infection, but not propagation

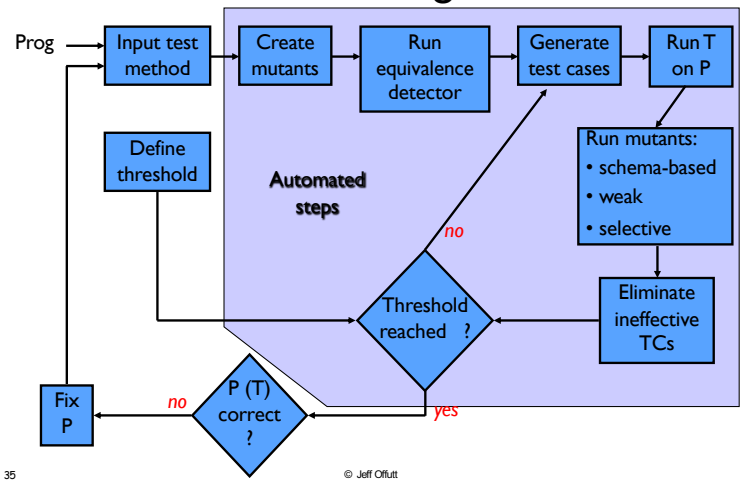
Weak Mutation Example

- Mutant 1 in the Min( ) example is:

```
     minVal = A;
Δ 1  minVal = B;
     if (B < A)
        minVal = B;
```

- The complete test specification to kill mutant 1:
- Reachability : *true* // Always get to that statement
- Infection : $A \neq B$
- Propagation: $(B < A) = false$ // Skip the next assignment
- Full Test Specification : $true \wedge (A \neq B) \wedge ((B < A) = false)$
  $\equiv (A \neq B) \wedge (B \geq A)$
  $\equiv (B > A)$
- Weakly kill mutant 1, but not strongly? A = 5, B = 3

Mutation Testing Process



Self-Check Questions

1. What does strong mutation require that weak mutation does not?
2. What do we call infeasible test requirements in mutation testing?
3. Do mutation operators (a) mimic programmer mistakes, (b) encourage common test heuristics, (c) both, or (d) something else?
4. Does mutation testing (a) evaluate tests, (b) help testers design tests, (c) both, or (d) neither?

Self-Check Answers

1. What does strong [Propagation] e that weak mutation does not?
2. What do we call in[Equivalent]uirements in mutation testing?
3. Do mutation operators (a) mimic programmer mistakes, (b) enco[Both]test heuristics, (c) both, or (d) something else?
4. Does mutation testi[Both]tests, (b) help testers design test[illegible]) neither?

Why Mutation Works

Fundamental Premise of Mutation Testing

If the software contains a fault, there will usually be a set of mutants that can only be killed by a test case that also detects that fault

- This is not an absolute!
- The mutants guide the tester to an effective set of tests
- A very challenging problem:
  - Find a fault and a set of mutation-adequate tests that do not find the fault
- Of course, this depends on the mutation operators …

In Practice

Practical Mutation Testing at Scale

A view from Google

Goran Petrović, Marko Ivanković, Gordon Fraser, René Just

Abstract—Mutation analysis assesses a test suite's adequacy by measuring its ability to detect small artificial faults, systematically seeded into the tested program. Mutation analysis is considered one of the strongest test-adequacy criteria. Mutation testing builds on top of mutation analysis and is a testing technique that uses mutants as test goals to create or improve a test suite. Mutation testing has long been considered intractable because the sheer number of mutants that can be created represents an insurmountable problem—both in terms of human and computational effort. This has hindered the adoption of mutation testing as an industry standard. For example, Google has a codebase of two billion lines of code and more than 150,000,000 tests are executed on a daily basis. The traditional approach to mutation testing does not scale to such an environment; even existing solutions to speed up mutation analysis are insufficient to make it computationally feasible at such a scale.
To address these challenges, this paper presents a scalable approach to mutation testing based on the following main ideas: (1) mutation testing is done incrementally, mutating only *changed code* during code review, rather than the entire code base; (2) mutants are filtered, removing mutants that are likely to be irrelevant to developers, and limiting the number of mutants per line and per code review process; (3) mutants are selected based on the historical performance of mutation operators, further eliminating irrelevant mutants and improving mutant quality. This paper empirically validates the proposed approach by analyzing its effectiveness in a code-review-based setting, used by more than 24,000 developers on more than 1,000 projects. The results show that the proposed approach produces orders of magnitude fewer mutants and that context-based mutant filtering and selection improve mutant quality and actionability. Overall, the proposed approach represents a mutation testing framework that seamlessly integrates into the software development workflow and is applicable to industrial settings of any size.

And at Facebook

What It Would Take to Use Mutation Testing in Industry—A Study at Facebook

Moritz Beller,* Chu-Pan Wong,‡ Johannes Bader,§ Andrew Scott,* Mateusz Machalica,* Satish Chandra,* Erik Meijer*
*Probability, Facebook, Inc., Menlo Park, USA
{mmb, andrewscott, msm, satch, erikm}@fb.com
‡Carnegie Mellon University (work done while interning at Facebook), Pittsburgh, USA
chupanw@cs.cmu.edu
§Jane Street Capital (ex-employee; work done while at Facebook)
johannes-bader@hotmail.de

an 2021

***Abstract*—Traditionally, mutation testing generates an abun-**

Designing Mutation Operators

- At the method level, mutation operators for different programming languages are similar
- Mutation operators do one of two things:
  - Mimic typical programmer mistakes (incorrect variable name)
  - Encourage common test heuristics (cause expressions to be 0)
- Researchers design lots of operators, then experimentally *select* the most useful
- Effective mutation operators yield mutants that are hard to kill, but not impossible
  - Tests that kill mutants from effective operators will also kill most other mutants

Mutation Operators for muJava

1. ABS — Absolute Value Insertion
2. AOR — Arithmetic Operator Replacement
3. ROR — Relational Operator Replacement
4. COR — Conditional Operator Replacement
5. SOR — Shift Operator Replacement
6. LOR — Logical Operator Replacement
7. ASR — Assignment Operator Replacement
8. UOI — Unary Operator Insertion
9. UOD — Unary Operator Deletion
10. SVR — Scalar Variable Replacement
11. BSR — Bomb Statement Replacement

Mutation Operators for Java

1. ABS Absolute Value Insertion:

Each arithmetic expression (and subexpression) is modified by the functions *abs()*, *negAbs()*, and *failOnZero()*.

Examples:

```
     a = m * (o + p);
Δ1   a = abs (m * (o + p));
Δ2   a = m * abs ((o + p));
Δ3   a = failOnZero (m * (o + p));
```

2. AOR Arithmetic Operator Replacement:

Each occurrence of one of the arithmetic operators +, −, *, /, and % is replaced by each of the other operators. In addition, each is replaced by the special mutation operators *leftOp*, and *rightOp*.

Examples:

```
     a = m * (o + p);
Δ1   a = m + (o + p);
Δ2   a = m * (o * p);
Δ3   a = m leftOp (o + p);
```


Mutation Operators for Java

3. ROR Relational Operator Replacement:

Each occurrence of one of the relational operators (<, ≤, >, ≥, =, ≠) is replaced by each of the other operators and by *falseOp* and *trueOp*.

Examples:

```
     if (X <= Y)
Δ1   if (X > Y)
Δ2   if (X < Y)
Δ3   if (X falseOp Y)  // always returns false
```

4. COR Conditional Operator Replacement:

Each occurrence of one of the logical operators (and - &&, or - || , and with no conditional evaluation - &, or with no conditional evaluation - |, not equivalent - ^) is replaced by each of the other operators; in addition, each is replaced by *falseOp*, *trueOp*, *leftOp*, and *rightOp*.

Examples:

```
     if (X <= Y && a > 0)
Δ1   if (X <= Y || a > 0)
Δ2   if (X <= Y leftOp a > 0) // returns result of left clause
```

Mutation Operators for Java

5. SOR Shift Operator Replacement:

Each occurrence of one of the shift operators <<, >>, and >>> is replaced by each of the other operators. In addition, each is replaced by the special mutation operator *leftOp*.

Examples:

```
     byte b = (byte) 16;
     b = b >> 2;
Δ1   b = b << 2;
Δ2   b = b leftOp 2; // result is b
```

6. LOR Logical Operator Replacement:

Each occurrence of one of the logical operators (bitwise and - &, bitwise or - |, exclusive or - ^) is replaced by each of the other operators; in addition, each is replaced by leftOp and rightOp.

Examples:

```
     int a = 60;   int b = 13;
     int c = a & b;
Δ1   int c = a | b;
Δ2   int c = a rightOp b; // result is b
```


Mutation Operators for Java

7. ASR Assignment Operator Replacement:

Each occurrence of one of the assignment operators (=, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=) is replaced by each of the other operators.

Examples:

```
     a = m * (o + p);
Δ1   a += m * (o + p);
Δ2   a *= m * (o + p);
```

8. UOI Unary Operator Insertion:

Each unary operator (arithmetic +, arithmetic -, conditional !, logical ~) is inserted in front of each expression of the correct type.

Examples:

```
     a = m * (o + p);
Δ1   a = m * -(o + p);
Δ2   a = -(m * (o + p));
```


Mutation Operators for Java

9. UOD Unary Operator Deletion:

Each unary operator (arithmetic +, arithmetic -, conditional !, logical~) is deleted.

Examples:

```
     if !(X <= Y && !Z)
Δ1   if (X > Y && !Z)
Δ2   if !(X < Y && Z)
```

10. SVR Scalar Variable Replacement:

Each variable reference is replaced by every other variable of the appropriate type that is declared in the current scope.

Examples:

```
     a = m * (o + p);
Δ 1  a = o * (o + p);
Δ 2  a = m * (m + p);
Δ 3  a = m * (o + o);
Δ 4  p = m * (o + p);
```


Mutation Operators for Java

11. BSR Bomb Statement Replacement:

Each statement is replaced by a special Bomb() function.

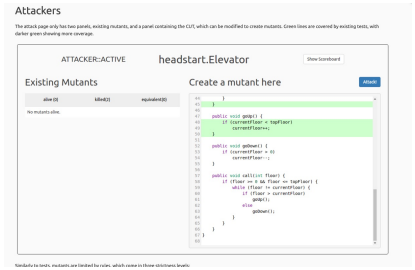
Example:

```
     a = m * (o + p);
Δ1   Bomb() // Raises exception when reached
```

Code Defenders

- A mutation “game”
  - by Gordon Fraser and José Miguel Roja, University of Sheffield
- Two players start with a Java class under test
  - Attacker creates a mutant of the class
  - Defender creates tests to try to kill the mutant

https://www.code-defenders.org/



Mutation Testing Tool for Assignment

https://pitest.org